PROIECT DE LABORATOR — TESTAREA SISTEMELOR SOFTWARE (TSS)

Sistemul de Control Acces într-o Parcare Corporate Multi-Tenant (`src/parking_gate.py`) · Universitatea din București · FMI · 2026

Student 1 — Natalia Moroz (50% contribuție):
• **Cerința 1:** Generarea datelor de test funcționale (EP: 15 teste, BVA: 25 teste, Graf Cauză-Efect: 9 teste) în `pytest`.
• **Cerința 2:** Măsurarea acoperirii la nivel de instrucțiune și de ramură pe seturile de la Cerința 1.
• **Cerința 5:** Formularea și demonstrarea unui mutant de ordinul 1 echivalent.

Student 2 — Victor Mura (50% contribuție):
• **Cerința 3:** Construirea grafului orientat de control (CFG), calculul complexității ciclomatice *V(G) = 7* și extragerea căilor independente.
• **Cerința 4:** Generarea setului minimal de teste pentru criteriul **MC/DC** pe decizia compusă EV.
• **Cerințele 6 & 7:** Doi mutanți ne-echivalenți (omorât vs. supraviețuitor) și analiza comparativă `mutmut` vs. `cosmic-ray`.

0. Specificația Problematicii și Codul Sursă Supus Testării

Funcția `evaluate_parking_access()` primește 7 parametri de intrare (cota companiei `company_allocated_quota`, locurile ocupate `company_current_occupied`, ora `entry_hour`, statutul angajatului `tier`, indicatorul `is_electric_vehicle`, disponibilitatea stațiilor EV `has_ev_spots_available` și permisiunea de depășire contra cost `company_allows_overflow`) și returnează decizia barierei (una din cele 6 valori `AccessDecision` sau excepția `ValueError`):

```
"""
Sistem de Control Acces Parcare Corporate Multi-Tenant (Smart Parking Gate).
Modulul principal implementează logica decizională pentru accesul vehiculelor
aparținând companiilor chiriașe într-un campus de afaceri.
"""

from enum import Enum, auto


class EmployeeTier(Enum):
    REGULAR = auto()      # Angajat standard
    VIP = auto()          # Management / Protocol
    CONTRACTOR = auto()   # Subcontractor extern / Temporar


class AccessDecision(Enum):
    GRANTED_REGULAR = auto()         # Acces gratuit în limita cotei companiei
    GRANTED_VIP = auto()             # Acces garantat necondiționat pe loc VIP
    GRANTED_EV_CHARGING = auto()     # Acces redirecționat la stația de încărcare EV
    GRANTED_PAID_OVERFLOW = auto()   # Acces suplimentar aprobat cu tarifare overflow
    DENIED_QUOTA_EXCEEDED = auto()   # Acces refuzat - cota companiei este epuizată
    DENIED_RESTRICTED_HOURS = auto() # Acces refuzat - în afara programului permis


def evaluate_parking_access(
    company_allocated_quota: int,
    company_current_occupied: int,
    entry_hour: int,
    tier: EmployeeTier,
    is_electric_vehicle: bool,
    has_ev_spots_available: bool,
    company_allows_overflow: bool,
) -> AccessDecision:
    """
    Evaluează cererea de acces a unui vehicul la bariera campusului corporate.

    :param company_allocated_quota: Numărul de locuri contractate de companie (> 0).
    :param company_current_occupied: Numărul curent de locuri ocupate de companie (>= 0).
    :param entry_hour: Ora curentă de intrare pe ceas (interval valid [0, 23]).
    :param tier: Statutul / rolul angajatului (EmployeeTier).
    :param is_electric_vehicle: True dacă vehiculul este complet electric / plug-in hybrid.
    :param has_ev_spots_available: True dacă există cel puțin un punct de încărcare EV liber.
    :param company_allows_overflow: True dacă firma permite depășirea cotei contra cost.
    :return: AccessDecision decizia automată a barierei.
    :raises ValueError: Dacă parametrii numerici sunt în afara domeniilor valide admise.
    """
```

```
47 |     # 0. Validare defensivă a parametrilor de intrare (Boundary Value / Equivalence)
48 |     if (
49 |         company_allocated_quota <= 0
50 |         or company_current_occupied < 0
51 |         or entry_hour < 0
52 |         or entry_hour > 23
53 |     ):
54 |         raise ValueError("Parametri de intrare în afara domeniilor valide admise.")
55 |
56 |     # Ore de vârf / program normal de lucru al campusului: 08:00 - 17:59
57 |     is_peak_hours = (8 <= entry_hour <= 17)
58 |
59 |     # 1. Prioritate VIP: Acces garantat necondiționat pe locuri rezervate de protocol
60 |     if tier == EmployeeTier.VIP:
61 |         return AccessDecision.GRANTED_VIP
62 |
63 |     # 2. Securitate Subcontractori: Acces interzis în afara orelor de program (noaptea)
64 |     if tier == EmployeeTier.CONTRACTOR and not is_peak_hours:
65 |         return AccessDecision.DENIED_RESTRICTED_HOURS
66 |
67 |     # 3. Stații de încărcare vehicule electrice (EV) - Condiție compusă (Nucleu MC/DC)
68 |     is_within_quota = company_current_occupied < company_allocated_quota
69 |     if is_electric_vehicle and has_ev_spots_available and (not is_peak_hours or is_within_quota):
70 |         return AccessDecision.GRANTED_EV_CHARGING
71 |
72 |     # 4. Acces standard în cota gratuită a companiei
73 |     if is_within_quota:
74 |         return AccessDecision.GRANTED_REGULAR
75 |
76 |     # 5. Politica de depășire cotă (Overflow plătit pe companie)
77 |     overflow_count = company_current_occupied - company_allocated_quota
78 |     if company_allows_overflow and (not is_peak_hours or overflow_count < 5):
79 |         return AccessDecision.GRANTED_PAID_OVERFLOW
80 |
81 |     # 6. Cota epuizată și overflow nepermis / indisponibil
82 |     return AccessDecision.DENIED_QUOTA_EXCEEDED
```

1. Cerința 1 — Testare Funcțională (Natalia Moroz)

1.1 Partiționarea în Clase de Echivalență (Equivalence Partitioning — EP)

Pentru partiționarea domeniului în clase de echivalență, identificăm mai întâi **clasele de ieșire (I_1 … I_7)** și **clasele de intrare (valide și invalide)** pentru fiecare parametru sau grup de parametri corelați, după care construim **clasele de echivalență finale (EC_1 … EC_{13})** ca asocieri între combinațiile de clase de intrare și clasa de ieșire produsă.

Clasă	Valoare / Excepție	Descriere
I1	**GRANTED_VIP**	Acces garantat necondiționat pe loc VIP (tier == VIP)
I2	**DENIED_RESTRICTED_HOURS**	Acces interzis pentru subcontractori în afara orelor 08:00–17:59
I3	**GRANTED_EV_CHARGING**	Acces aprobat cu alocarea unei stații de încărcare EV
I4	**GRANTED_REGULAR**	Acces gratuit în limita cotei alocate companiei (occupied < quota)
I5	**GRANTED_PAID_OVERFLOW**	Acces suplimentar cu plată (overflow permis și off-peak sau depășire < 5)
I6	**DENIED_QUOTA_EXCEEDED**	Acces refuzat — cota companiei este epuizată și overflow indisponibil
I7	**ValueError**	Excepție ridicată pentru parametri numerici în afara domeniului valid

Parametru	Clase de Intrare Valide	Clase Invalide
quota (q)	Q1: q ≥ 1	Q2: q = 0 · Q3: q < 0
occupied (o)	O1: 0 ≤ o < q · O2: o = q · O3: o > q	O4: o < 0
entry_hour (h)	H1: 0 ≤ h ≤ 7 · H2: 8 ≤ h ≤ 17 · H3: 18 ≤ h ≤ 23	H4: h < 0 · H5: h > 23
tier (t)	T1: VIP · T2: REGULAR · T3: CONTRACTOR	—
(is_ev, has_spot)	V1: (T, T) · V2: (T, F) · V3: (F, —)	—
allows_overflow	P1: True · P2: False	—

Selecția claselor pentru `(is_ev, has_spot)`: Când `is_ev = True`, disponibilitatea stației schimbă decizia între I_3 și fluxul standard, deci avem două clase: V_1 = `(T, T)` și V_2 = `(T, F)`. Când `is_ev = False`, conjuncția `is_electric_vehicle and has_ev_spots_available` este `False` indiferent de `has_spot`, deci `(F, T)` și `(F, F)` sunt echivalente comportamental și formează clasa V_3 = `(F, —)`.

Tabelul Claselor de Echivalență Finale (EC_1 ... EC_{13}) și Valorile Testate (`tests/test_ep.py`)

Clasă Finală	Combinație Clase de Intrare	Clasă Ieșire	Valoare Testată (`q, o, h, tier, ev, spot, ovf`)	Rezultat Obținut
EC1	Q2 (restul valide)	**I7**	(0, 0, 10, REGULAR, F, F, F)	**ValueError**
EC2	Q3 (restul valide)	**I7**	(−10, 0, 10, REGULAR, F, F, F)	**ValueError**
EC3	O4 (restul valide)	**I7**	(10, −1, 10, REGULAR, F, F, F)	**ValueError**
EC4	H4 (restul valide)	**I7**	(10, 5, −5, REGULAR, F, F, F)	**ValueError**
EC5	H5 (restul valide)	**I7**	(10, 5, 24, REGULAR, F, F, F)	**ValueError**
EC5	H5 (restul valide)	**I7**	(10, 5, 30, REGULAR, F, F, F)	**ValueError**
EC6	Q1, O2, H3, T1, V3, P2	**I1**	(10, 10, 22, VIP, F, F, F)	**GRANTED_VIP**
EC7	Q1, O1, H3, T3, V3, P2	**I2**	(10, 2, 19, CONTRACTOR, F, F, F)	**DENIED_RESTRICTED_HOURS**
EC8	Q1, O2, H1, T2, V1, P2	**I3**	(10, 10, 6, REGULAR, T, T, F)	**GRANTED_EV_CHARGING**
EC9	Q1, O1, H2, T2, V3, P2	**I4**	(10, 5, 11, REGULAR, F, F, F)	**GRANTED_REGULAR**
EC10	Q1, O1, H2, T3, V3, P2	**I4**	(10, 2, 10, CONTRACTOR, F, F, F)	**GRANTED_REGULAR**
EC11	Q1, O1, H2, T2, V2, P2	**I4**	(10, 5, 11, REGULAR, T, F, F)	**GRANTED_REGULAR**
EC12	Q1, O3, H2, T2, V3, P1	**I5**	(10, 12, 11, REGULAR, F, F, T)	**GRANTED_PAID_OVERFLOW**
EC13	Q1, O2, H2, T2, V3, P2	**I6**	(10, 10, 11, REGULAR, F, F, F)	**DENIED_QUOTA_EXCEEDED**

1.2 Analiza Valorilor de Frontieră (Boundary Value Analysis — BVA)

Pentru fiecare frontieră numerică a domeniilor de intrare și a pragurilor decizionale am testat tripletul `(frontieră − 1, frontieră, frontieră + 1)`. Mai jos sunt afișate **toate cele 25 de cazuri BVA** implementate în `tests/test_bva.py`:

Caz #	Frontieră Testată	Valoare Testată `(q, o, h, tier, ev, spot, ovf)`	Clasă Ieșire	Rezultat Obținut
Frontieră: quota ≥ 1 (variabila q)				
B01	quota ≥ 1	`(0, 0, 12, REGULAR, F, F, F)`	**I7**	**ValueError**
B02	quota ≥ 1	`(1, 0, 12, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B03	quota ≥ 1	`(2, 0, 12, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
Frontieră: occupied ≥ 0 (variabila o)				
B04	occupied ≥ 0	`(10, −1, 12, REGULAR, F, F, F)`	**I7**	**ValueError**
B05	occupied ≥ 0	`(10, 0, 12, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B06	occupied ≥ 0	`(10, 1, 12, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
Frontieră: occupied < quota (variabila o)				
B07	occupied < quota	`(10, 9, 12, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B08	occupied < quota	`(10, 10, 12, REGULAR, F, F, F)`	**I6**	**DENIED_QUOTA_EXCEEDED**
B09	occupied < quota	`(10, 11, 12, REGULAR, F, F, F)`	**I6**	**DENIED_QUOTA_EXCEEDED**
Frontieră: hour ≥ 0 (variabila h)				
B10	hour ≥ 0	`(10, 5, −1, REGULAR, F, F, F)`	**I7**	**ValueError**
B11	hour ≥ 0	`(10, 5, 0, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B12	hour ≥ 0	`(10, 5, 1, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
Frontieră: vârf: h ≥ 8 (variabila h)				
B13	vârf: h ≥ 8	`(10, 5, 7, CONTRACTOR, F, F, F)`	**I2**	**DENIED_RESTRICTED_HOURS**
B14	vârf: h ≥ 8	`(10, 5, 8, CONTRACTOR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B15	vârf: h ≥ 8	`(10, 5, 9, CONTRACTOR, F, F, F)`	**I4**	**GRANTED_REGULAR**
Frontieră: vârf: h ≤ 17 (variabila h)				
B16	vârf: h ≤ 17	`(10, 5, 16, CONTRACTOR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B17	vârf: h ≤ 17	`(10, 5, 17, CONTRACTOR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B18	vârf: h ≤ 17	`(10, 5, 18, CONTRACTOR, F, F, F)`	**I2**	**DENIED_RESTRICTED_HOURS**
Frontieră: hour ≤ 23 (variabila h)				
B19	hour ≤ 23	`(10, 5, 22, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B20	hour ≤ 23	`(10, 5, 23, REGULAR, F, F, F)`	**I4**	**GRANTED_REGULAR**
B21	hour ≤ 23	`(10, 5, 24, REGULAR, F, F, F)`	**I7**	**ValueError**
Frontieră: depășire o − q < 5 (variabila o−q)				
B22	depășire o − q < 5	`(10, 14, 10, REGULAR, F, F, T)`	**I5**	**GRANTED_PAID_OVERFLOW**
B23	depășire o − q < 5	`(10, 15, 10, REGULAR, F, F, T)`	**I6**	**DENIED_QUOTA_EXCEEDED**
B24	depășire o − q < 5	`(10, 16, 10, REGULAR, F, F, T)`	**I6**	**DENIED_QUOTA_EXCEEDED**
Frontieră: caz special (q ≠ 10) (variabila o−q)				
B25	caz special (q ≠ 10)	`(4, 11, 10, REGULAR, F, F, T)`	**I6**	**DENIED_QUOTA_EXCEEDED**

1.3 Graful Cauză-Efect (Cause-Effect Graph — CEG)

Pe domeniul valid al datelor de intrare, logica funcției este modelată prin **8 cauze booleene (C_1 … C_8)**, **3 noduri intermediare numerotate 1, 2, 3** și **6 efecte (Ef_1 … Ef_6)**:

Cauză	Semnificație Booleană
C1	tier = VIP
C2	tier = CONTRACTOR
C3	8 ≤ h ≤ 17 (ore de vârf)
C4	vehicul electric
C5	stație EV liberă
C6	o < q (în cotă)
C7	overflow permis
C8	o − q < 5

Efect	Ieșire	Decizie	Expresie din Graf
Ef1	I1	GRANTED_VIP	C1
Ef2	I2	DENIED_RESTRICTED_HOURS	C2 ∧ ¬C3
Ef3	I3	GRANTED_EV_CHARGING	C4 ∧ C5 ∧ (¬C3 ∨ C6)
Ef4	I4	GRANTED_REGULAR	C6
Ef5	I5	GRANTED_PAID_OVERFLOW	¬C6 ∧ C7 ∧ (¬C3 ∨ C8)
Ef6	I6	DENIED_QUOTA_EXCEEDED	¬C6 ∧ ¬(C7 ∧ (¬C3 ∨ C8))

Clarificarea Notațiilor și a Relațiilor de Mascare (M) între Efecte:
• **Eliminarea confuziei I_1, I_2, I_3:** În secțiunea 1.1, notația I_1 … I_7 reprezintă *clasele de ieșire*. De aceea, nodurile intermediare din graf sunt notate cu cifrele **1, 2, 3** în cercuri: nodul 1 = $\neg C_3 \vee C_6$, nodul 2 = $\neg C_3 \vee C_8$, nodul 3 = $C_7 \wedge$ nodul 2.
• **Constrângerea de exclusivitate pe cauze $E(C_1, C_2)$:** Un angajat nu poate avea simultan rolul VIP (C_1=1) și CONTRACTOR (C_2=1).
• **Relații de mascare (M) între efecte:** Deoarece instrucțiunile `if ... return` se evaluează secvențial, activarea unui efect superior maschează efectele de mai jos (le forțează la 0): $M(Ef_1, \{Ef_3, Ef_4, Ef_5, Ef_6\})$, $M(Ef_2, \{Ef_3, Ef_4, Ef_5, Ef_6\})$ și $M(Ef_3, \{Ef_4, Ef_5, Ef_6\})$. Efectele Ef_4, Ef_5, Ef_6 sunt deja reciproc disjuncte prin condițiile C_6 vs. $\neg C_6$ și nodul 3 vs. ¬nodul 3.

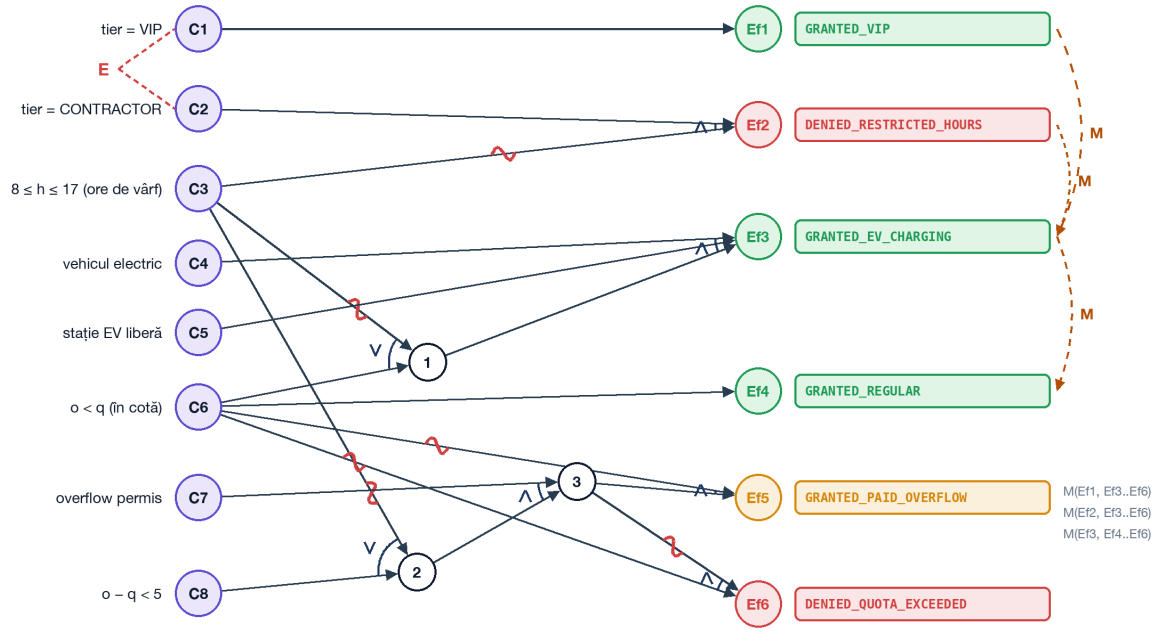


Figura 1 — Graful Cauză-Efect realizat în Draw.io (`docs/CEG_Drawio.drawio`). Negația este marcată prin simbolul ~ pe muchie, conjuncția/disjuncția prin arcele cu ∧ și ∨, exclusivitatea cauzelor prin E, iar mascarea între efecte prin M.

1.4 Construcția Pas cu Pas a Tabelului de Decizie până la Forma Finală

Pornind de la fiecare efect $Ef_k = 1$ ($k = 1 \dots 6$), parcurgem graful invers către cauze și adăugăm coloanele (regulile) corespunzătoare, ținând cont de exclusivitatea $E(C_1, C_2)$ și de măștile M:

Pasul 1 — Efectul Ef1 = 1 (GRANTED_VIP)

`Ef1 = C1`

Ef1 = C1 = 1 ⇒ C1 = 1; din E(C1, C2) ⇒ C2 = 0.
Restul cauzelor nu influențează efectul; în cazul ales expresia lui Ef6 ar fi și ea 1,
dar M(Ef1, Ef6) o forțează la 0.

	R1
C1	1
C2	0
C3	1
C4	1
C5	1
C6	0
C7	0
C8	1
Ef1	1
Ef2	0
Ef3	0
Ef4	0
Ef5	0
Ef6	0

Pasul 2 — Efectul Ef2 = 1 (DENIED_RESTRICTED_HOURS)

`Ef2 = C2 ∧ ¬C3`

Ef2 = C2 ∧ ¬C3 = 1 ⇒ C2 = 1, C3 = 0; din E ⇒ C1 = 0.
Fără EV, sub cotă: Ef4 ar fi 1, dar este mascat de Ef2.

	R1	R2
C1	1	0
C2	0	1
C3	1	0
C4	1	0
C5	1	0
C6	0	1
C7	0	0
C8	1	1
Ef1	1	0
Ef2	0	1
Ef3	0	0
Ef4	0	0
Ef5	0	0
Ef6	0	0

Pasul 3 — Efectul Ef3 = 1 (GRANTED_EV_CHARGING)

`Ef3 = C4 ∧ C5 ∧ (¬C3 ∨ C6)`

Ef3 = C4 ∧ C5 ∧ (¬C3 ∨ C6) = 1 ⇒ C4 = C5 = 1 și (C3 = 0 sau C6 = 1).
Nodul ∨ are două combinații minimale ⇒ două coloane: R3 (C3 = 0) și R4 (C3 = 1, C6 = 1).
C1 = C2 = 0 (altfel Ef1/Ef2 ar masca Ef3).

	R1	R2	R3	R4
C1	1	0	0	0
C2	0	1	0	0
C3	1	0	0	1
C4	1	0	1	1
C5	1	0	1	1
C6	0	1	0	1
C7	0	0	0	0
C8	1	1	1	1
Ef1	1	0	0	0
Ef2	0	1	0	0
Ef3	0	0	1	1
Ef4	0	0	0	0
Ef5	0	0	0	0
Ef6	0	0	0	0

Pasul 4 — Efectul Ef4 = 1 (GRANTED_REGULAR)

`Ef4 = C6 (cu Ef1..Ef3 = 0)`

Ef4 = C6 = 1, iar C1 = C2 = 0 și Ef3 = 0 (C4 = 0).

	R1	R2	R3	R4	R5
C1	1	0	0	0	0
C2	0	1	0	0	0
C3	1	0	0	1	1
C4	1	0	1	1	0
C5	1	0	1	1	0
C6	0	1	0	1	1
C7	0	0	0	0	0
C8	1	1	1	1	1
Ef1	1	0	0	0	0
Ef2	0	1	0	0	0
Ef3	0	0	1	1	0
Ef4	0	0	0	0	1
Ef5	0	0	0	0	0
Ef6	0	0	0	0	0

Pasul 5 — Efectul Ef5 = 1 (GRANTED_PAID_OVERFLOW)

$Ef5 = \neg C6 \wedge C7 \wedge (\neg C3 \vee C8)$

Ef5 = ¬C6 ∧ C7 ∧ (¬C3 ∨ C8) = 1 ⇒ C6 = 0, C7 = 1 și (C3 = 0 sau C8 = 1).
Două combinații ⇒ R6 (noapte) și R7 (vârf, depășire < 5).

	R1	R2	R3	R4	R5	R6	R7
C1	1	0	0	0	0	0	0
C2	0	1	0	0	0	0	0
C3	1	0	0	1	1	0	1
C4	1	0	1	1	0	0	0
C5	1	0	1	1	0	0	0
C6	0	1	0	1	1	0	0
C7	0	0	0	0	0	1	1
C8	1	1	1	1	1	0	1
Ef1	1	0	0	0	0	0	0
Ef2	0	1	0	0	0	0	0
Ef3	0	0	1	1	0	0	0
Ef4	0	0	0	0	1	0	0
Ef5	0	0	0	0	0	1	1
Ef6	0	0	0	0	0	0	0

Pasul 6 — Efectul Ef6 = 1 (DENIED_QUOTA_EXCEEDED)

$Ef6 = \neg C6 \wedge \neg(C7 \wedge (\neg C3 \vee C8))$

Ef6 = ¬C6 ∧ ¬(C7 ∧ (¬C3 ∨ C8)) = 1 ⇒ C6 = 0 și nodul 3 = 0:
fie C7 = 1, C3 = 1, C8 = 0 (R8), fie C7 = 0 (R9).

	R1	R2	R3	R4	R5	R6	R7	R8	R9
C1	1	0	0	0	0	0	0	0	0
C2	0	1	0	0	0	0	0	0	0
C3	1	0	0	1	1	0	1	1	1
C4	1	0	1	1	0	0	0	0	0
C5	1	0	1	1	0	0	0	0	0
C6	0	1	0	1	1	0	0	0	0
C7	0	0	0	0	0	1	1	1	0
C8	1	1	1	1	1	0	1	0	1
Ef1	1	0	0	0	0	0	0	0	0
Ef2	0	1	0	0	0	0	0	0	0
Ef3	0	0	1	1	0	0	0	0	0
Ef4	0	0	0	0	1	0	0	0	0
Ef5	0	0	0	0	0	1	1	0	0
Ef6	0	0	0	0	0	0	0	1	1

Tabelul de Decizie Final (R_1 ... R_9) cu Valori de Intrare și Ieșire (`tests/test_ceg.py`)

Cauză / Efect	R1	R2	R3	R4	R5	R6	R7	R8	R9
C1: tier = VIP	1	0	0	0	0	0	0	0	0
C2: tier = CONTRACTOR	0	1	0	0	0	0	0	0	0
C3: 8 ≤ h ≤ 17 (ore de vârf)	1	0	0	1	1	0	1	1	1
C4: vehicul electric	1	0	1	1	0	0	0	0	0
C5: stație EV liberă	1	0	1	1	0	0	0	0	0
C6: o < q (în cotă)	0	1	0	1	1	0	0	0	0
C7: overflow permis	0	0	0	0	0	1	1	1	0
C8: o − q < 5	1	1	1	1	1	0	1	0	1
Efecte (Ef1 .. Ef6) după aplicarea măștilor M									
Ef1: GRANTED_VIP	1	0	0	0	0	0	0	0	0
Ef2: DENIED_RESTRICTED_HOURS	0	1	0	0	0	0	0	0	0
Ef3: GRANTED_EV_CHARGING	0	0	1	1	0	0	0	0	0
Ef4: GRANTED_REGULAR	0	0	0	0	1	0	0	0	0
Ef5: GRANTED_PAID_OVERFLOW	0	0	0	0	0	1	1	0	0
Ef6: DENIED_QUOTA_EXCEEDED	0	0	0	0	0	0	0	1	1
Input (q, o, h, tier, ev, sp, ovf)	(10,10,14,VIP,T,T,F)	(10,2,21,CON,F,F,F)	(10,10,5,REG,T,T,F)	(10,7,10,REG,T,T,F)	(10,8,10,REG,F,F,F)	(10,20,23,REG,F,F,T)	(10,13,11,REG,F,F,T)	(10,16,11,REG,F,F,T)	(10,10,11,REG,F,F,F)
Output (decizie barieră)	I1: VIP	I2: RESTRICTED_HOURS	I3: EV_CHARGING	I3: EV_CHARGING	I4: REGULAR	I5: PAID_OVERFLOW	I5: PAID_OVERFLOW	I6: QUOTA_EXCEEDED	I6: QUOTA_EXCEEDED

2. Cerința 2 — Analiza Acoperirii Codului pe Seturile de la Cerința 1 (Natalia Moroz)

Diferența dintre Acoperirea la Nivel de Instrucțiune (Statement Coverage) și Acoperirea la Nivel de Ramură (Branch Coverage):
• **Acoperirea la nivel de instrucțiune (29 instrucțiuni în `src/parking_gate.py`):** Măsoară câte linii executabile au fost rulate cel puțin o dată. O instrucțiune `if cond:` este marcată ca executată chiar dacă expresia `cond` a luat doar valoarea `False`.
• **Acoperirea la nivel de ramură (12 ramuri = 6 instrucțiuni `if` × 2 ieșiri `True`/`False`):** Impune ca fiecare decizie `if` să fie parcursă atât pe ramura `True`, cât și pe ramura `False`. În `src/parking_gate.py`, neintrarea în corpul unui `if` lasă 1 instrucțiune din 29 neacoperită (–3.4%), dar lasă 1 ramură din 12 neacoperită (–8.3%).

Set Teste	Nr. Teste	Instrucțiuni Acoperite	Ramuri Acoperite	Total	Linii Lipsă	Interpretare și Motivul Neacoperirii
EP	15	`29 / 29 (100.0%)`	`12 / 12 (100.0%)`	**`100%`**	–	Fiecare ieșire I1…I7 apare în cel puțin o clasă finală ⇒ toate cele 12 ramuri sunt parcurse.
BVA	25	`27 / 29 (93.1%)`	`10 / 12 (83.3%)`	**`90%`**	`61, 70`	Frontierele sunt numerice; VIP (linia 61) și EV (linia 70) sunt condiții booleene fără frontieră, deci nu apar.
CEG	9	`28 / 29 (96.6%)`	`11 / 12 (91.7%)`	**`95%`**	`54`	Cauzele descriu doar intrări valide; ramura ValueError (linia 54) nu e un efect al grafului.

3. Cerința 3 — Graful Orientat de Control (CFG), Complexitatea Ciclomatică și Căile Independente (Victor Mura)

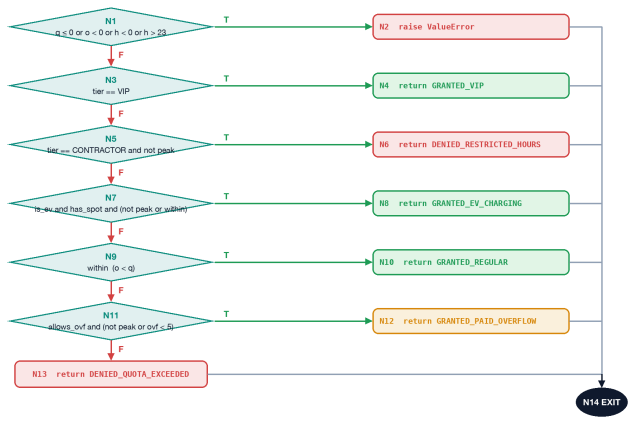


Figura 2 — Graful Orientat de Flux de Control (`docs/CFG_Drawio.drawio`)

Calculul Complexității Ciclomatice McCabe *V*(*G*)

Graful orientat are $n = 14$ noduri (6 noduri predicat `N1, N3, N5, N7, N9, N11`, 7 noduri de retur/excepție `N2, N4, N6, N8, N10, N12, N13` și nodul de ieșire N14) și $e = 19$ arce orientate:

1. **Formula cu arce și noduri:**
   $V(G) = e - n + 2 = 19 - 14 + 2 = \mathbf{7}$
   (sau $e_{ext} - n + 1 = 20 - 14 + 1 = \mathbf{7}$ incluzând arcul N14 → N1).
2. **Formula cu noduri predicat ($P = 6$):**
   $V(G) = P + 1 = 6 + 1 = \mathbf{7}$.
3. **Formula cu regiuni în plan:**
   $V(G) = R = 6$ regiuni închise + 1 regiune exterioară = **7**.

Cele 7 Căi Liniar Independente (P_1 … P_7)

Cale	Secvența de Noduri Parcursă	Intrare (`q, o, h, tier, ev, sp, ovf`)	Ieșire
P1	`N1 → N2 → N14`	`(0, 0, 10, REGULAR, F, F, F)`	**`I7: ValueError`**
P2	`N1 → N3 → N4 → N14`	`(10, 10, 22, VIP, F, F, F)`	**`I1: GRANTED_VIP`**
P3	`N1 → N3 → N5 → N6 → N14`	`(10, 2, 19, CONTRACTOR, F, F, F)`	**`I2: DENIED_RESTRICTED_HOURS`**
P4	`N1 → N3 → N5 → N7 → N8 → N14`	`(10, 10, 6, REGULAR, T, T, F)`	**`I3: GRANTED_EV_CHARGING`**
P5	`N1 → N3 → N5 → N7 → N9 → N10 → N14`	`(10, 5, 11, REGULAR, F, F, F)`	**`I4: GRANTED_REGULAR`**
P6	`N1 → N3 → N5 → N7 → N9 → N11 → N12 → N14`	`(10, 12, 11, REGULAR, F, F, T)`	**`I5: GRANTED_PAID_OVERFLOW`**
P7	`N1 → N3 → N5 → N7 → N9 → N11 → N13 → N14`	`(10, 10, 11, REGULAR, F, F, F)`	**`I6: DENIED_QUOTA_EXCEEDED`**

4. Cerința 4 — Criteriul MC/DC pe Decizia Compusă EV (Victor Mura)

Analizăm decizia compusă de la linia 69 din `src/parking_gate.py`, care conține $N = 4$ condiții atomice independente:

$D = (A \wedge B) \wedge (\neg C \vee Dq)$ unde: A = is_electric_vehicle, B = has_ev_spots_available, C = is_peak_hours, Dq = is_within_quota

Setul minimal care satisface criteriul MC/DC are $N + 1 = 5$ teste (`T1..T5` în `tests/test_mcdc.py`):

Test	A	B	C	Dq	D	Intrare (q, o, h, tier, ev, sp, ovf)	Rezultat	Pereche Independență
T1	1	1	0	0	**1**	(10, 10, 6, REG, T, T, F)	**GRANTED_EV_CHARGING**	Bază pt. A, B, C
T2	0	1	0	0	0	(10, 10, 6, REG, F, T, F)	DENIED_QUOTA_EXCEEDED	**A: (T1, T2)**
T3	1	0	0	0	0	(10, 10, 6, REG, T, F, F)	DENIED_QUOTA_EXCEEDED	**B: (T1, T3)**
T4	1	1	1	0	0	(10, 10, 10, REG, T, T, F)	DENIED_QUOTA_EXCEEDED	**C: (T1, T4)**
T5	1	1	1	1	**1**	(10, 7, 10, REG, T, T, F)	**GRANTED_EV_CHARGING**	**Dq: (T4, T5)**

5. Cerința 5 — Mutant de Ordinul 1 Echivalent (Natalia Moroz)

```
56 |     # Ore de vârf / program normal de lucru al campusului: 08:00 - 17:59
57 | -   is_peak_hours = (8 <= entry_hour <= 17)     # Programul original P
57 | +   is_peak_hours = (7 < entry_hour < 18)       # Mutantul echivalent M_eq (mutants/equivalent_mutant.py)
```

Demonstrație: Deoarece `entry_hour` este întreg ($h \in \mathbb{Z}$), pe mulțimea numerelor întregi avem echivalențele $h \geq 8 \Leftrightarrow h > 7$ și $h \leq 17 \Leftrightarrow h < 18$. Astfel, valoarea booleană a lui `is_peak_hours` este identică în P și în M_{eq} pentru orice $h \in \mathbb{Z}$, deci $P(x) = M_{eq}(x)$ pentru toate intrările posibile (verificat exhaustiv pe 1 152 combinații în `tests/test_equivalent_mutant.py`).

6. Cerința 6 — Doi Mutanți de Ordinul 1 Ne-echivalenți: Omorât vs. Neomorât (Victor Mura)

Ambii mutanți sunt evaluați față de același caz de test din Cerința 1 (BVA, B08): `test_bva_occupied_exactly_at_quota`, care apelează funcția cu intrarea:
`q = 10, o = 10, h = 12, tier = REGULAR, ev = False, spot = False, company_allows_overflow = False` și verifică `assert res == AccessDecision.DENIED_QUOTA_EXCEEDED`.

6.1 Mutantul #1 (Ne-echivalent, OMORÂT de `test_bva_occupied_exactly_at_quota`)

```
68 | -   is_within_quota = company_current_occupied < company_allocated_quota
68 | +   is_within_quota = company_current_occupied <= company_allocated_quota   # Mutație ROR: < înlocuit cu <=
...
73 |     if is_within_quota:
74 |         return AccessDecision.GRANTED_REGULAR
```

Pentru `(q=10, o=10)`, programul original calculează `10 < 10 → False` și returnează la final `DENIED_QUOTA_EXCEEDED`. Mutantul #1 calculează `10 <= 10 → True` și returnează la linia 74 `GRANTED_REGULAR`, deci aserțiunea testului eșuează și **mutantul este omorât**.

6.2 Mutantul #2 (Ne-echivalent, NEOMORÂT de `test_bva_occupied_exactly_at_quota`)

Mai jos este redată integral secvența liniilor 72–82 din `src/parking_gate.py` pentru a se vedea complet instrucțiunea mutată:

```
72 |     # 4. Acces standard în cota gratuită a companiei
73 |     if is_within_quota:                                  # (10 < 10) este False -> nu intră
74 |         return AccessDecision.GRANTED_REGULAR
75 |
76 |     # 5. Politica de depășire cotă (Overflow plătit pe companie)
77 |     overflow_count = company_current_occupied - company_allocated_quota   # 10 - 10 = 0
78 | -   if company_allows_overflow and (not is_peak_hours or overflow_count < 5):
78 | +   if company_allows_overflow and (not is_peak_hours or overflow_count < 4):  # MUTAȚIE: 5 -> 4
79 |         return AccessDecision.GRANTED_PAID_OVERFLOW
80 |
81 |     # 6. Cota epuizată și overflow nepermis / indisponibil
82 |     return AccessDecision.DENIED_QUOTA_EXCEEDED
```

De ce expresia mutată `overflow_count < 4` nu se evaluează în acest test:
• În `test_bva_occupied_exactly_at_quota`, parametrul `company_allows_overflow` este `False`.
• La linia 78, operatorul `and` din Python evaluează cu **scurtcircuitare (short-circuit evaluation)**: deoarece operandul din stânga (`company_allows_overflow`) este deja `False`, rezultatul conjuncției este garantat `False`, iar Python **nu mai execută deloc paranteza din dreapta `(not is_peak_hours or overflow_count < 4)`**.
• Prin urmare, atât originalul *P*, cât și Mutantul #2 trec direct la linia 82 și returnează `DENIED_QUOTA_EXCEEDED` (mutantul supraviețuiește acestui test).
• Mutantul #2 **nu este echivalent**: pentru testul BVA B22 `(q=10, o=14, h=10, REGULAR, F, F, company_allows_overflow=True)`, unde `overflow_count = 4` și `company_allows_overflow = True`, paranteza se execută și *P* returnează `GRANTED_PAID_OVERFLOW` (4 < 5), în timp ce M_2 returnează `DENIED_QUOTA_EXCEEDED` (4 < 4 este `False`).

7. Cerința 7 — Compararea Utilitarelor `mutmut` și `cosmic-ray` (Victor Mura)

Semnificația suitelor „Cerința 1” și „Cerința 4” în evaluarea mutațiilor:
• **Suita Cerinței 1 (49 teste funcționale):** Cuprinde testele Black-Box din `test_ep.py` (15), `test_bva.py` (25) și `test_ceg.py` (9).
• **Suita Cerinței 4 (5 teste structurale MC/DC):** Cuprinde doar cele 5 teste `T1..T5` din `test_mcdc.py` proiectate special pentru decizia EV de la linia 69.
• **Suita Combinată (Cerința 1 + Cerința 4 = 54 teste):** Reunește ambele seturi pentru a măsura capacitatea totală de detecție a defectelor.

Suită de Teste Rulată	`mutmut` (Mutanți Uciși / Total)	`cosmic-ray` (Mutanți Uciși / Total)
Suita Cerinței 1 — Teste funcționale (49 teste: EP 15 + BVA 25 + CEG 9)	**`39 / 40 (97.5%)`**	**`98 / 101 (97.03%)`**
Suita Cerinței 4 — Teste structurale MC/DC (5 teste: T1..T5 în test_mcdc.py)	**`10 / 40 (25.0%)`**	**`50 / 101 (49.5%)`**
Suita Combinată — Cerința 1 + Cerința 4 (54 teste)	**`40 / 40 (100.0%)`**	**`99 / 101 (98.02%)`**

Argumentarea diferențelor dintre cele două utilitare: `mutmut` generează **40 de mutanți** pe expresiile logice, relaționale și literale: 39/40 (97.5%) sunt uciși de suita Cerinței 1, 10/40 (25.0%) de suita Cerinței 4 (care vizează exclusiv decizia EV de la linia 69), și **40/40 (100.0%)** de suita combinată (Cerința 1 + Cerința 4). `cosmic-ray` aplică o gamă mai largă de operatori AST (**101 mutanți**), din care 98/101 (97.03%) sunt uciși de Cerința 1, 50/101 (49.5%) de Cerința 4 și **99/101 (98.02%)** de suita combinată; cei 2 mutanți supraviețuitori sunt mutanți echivalenți, rezultând un scor de mutație ajustat $MS = D / (M - E) = 100\%$.

8. Contribuția Echipei și Concluzii

Autor	Pondere	Activități Realizate și Contribuție Proprie
Natalia Moroz	**50%**	**Cerințele 1, 2, 5:** Determinarea claselor de intrare/ieșire și a celor 13 clase finale EP (`test_ep.py`), proiectarea celor 25 de cazuri BVA (`test_bva.py`), modelarea grafului Cauză-Efect în Draw.io (`CEG_Drawio.drawio`) cu relațiile de mascare *M* și derivarea pas cu pas a tabelului de decizie (`test_ceg.py`); analiza acoperirii pe instrucțiuni vs. ramuri; demonstrarea și verificarea exhaustivă a mutantului echivalent. Referințe tehnice auxiliare în `AI_USAGE_LAB.md`.
Victor Mura	**50%**	**Cerințele 3, 4, 6, 7:** Construirea grafului orientat de flux de control (`CFG_Drawio.drawio`), calculul complexității ciclomatice $V(G)=7$ prin 3 metode și extragerea celor 7 căi independente; determinarea perechilor de independență MC/DC și implementarea setului minimal de 5 teste (`test_mcdc.py`); proiectarea și analiza celor doi mutanți ne-echivalenți (omorât vs. supraviețuitor prin scurtcircuitare) și rularea comparativă `mutmut` vs. `cosmic-ray`.